

표적 궤적 모의

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그리기까지

원리 · 기동 모델 · C 구현 · 결과

레이다 시스템 소프트웨어 스터디 · Part 3

INDEX

목차

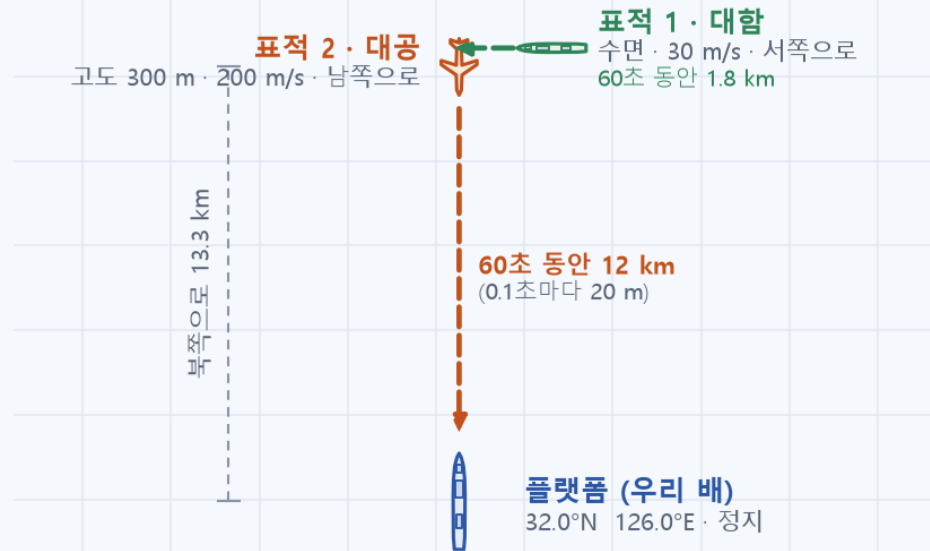
00	들어가며	과제 3이 묻는 것
01	표적을 움직이는 원리	좌표계 넷 · 위치와 속도 시간 적분 · 지구 곡률 · 한 스텝
02	기동 모델	하중배수 · 원운동 세 축 · 기동 시간표
03	C 로 구현	구성 · 구조체 · 과제 조건 속도 변환 · 한 스텝 · 화면
04	결과	조건 · 실행 화면 · 궤적 기동 시나리오 · 검증

수학은 고등학교 물리의 등속 원운동 정도면 충분하고, 나올 때마다 다시 설명합니다.

00. 과제 3이 묻는 것

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라

위에서 내려다본 그림 (대략 축척)



시뮬레이션 시간

60 초 · 0.1 초 간격

- $t = 0.0 \text{ s}$
- $t = 0.1 \text{ s}$
- $t = 0.2 \text{ s}$
- $t = 0.3 \text{ s}$
- $t = 60.0 \text{ s}$

위치 갱신 600 번
표적마다 · 위경도로 기록

① 표적 구조체를 정의한다

표적 최대 10개 · 표적마다 기동 최대 30개
초기 위치는 위경도·고도, 속도는 동체 속력 하나로

② 위치·속도는 ECEF 에서 갱신한다

동체 속력을 ECEF 성분으로 바꾸는 방법도 고민 (추가 항목)

③ 60초 동안 0.1초 간격으로

플랫폼 하나 (정지) · 대함 표적 · 대공 표적

④ 위경도로 출력하고 그림으로

플랫폼과 표적의 궤적을 지도 위에

과제 2 와 다른 점 : 변환 함수 넷은 제공 코드 그대로, 새로 들어온 것은 속도 · 시간 · 기동

지난번엔 측정값 하나가 체인을 한 번 지나 점 하나. 이번엔 같은 계산을 600번 반복해 선이 된다.

01

표적을 움직이는 원리

- 1 좌표계 넷, 그리고 왜 ECEF 인가
- 2 위치와 속도는 다르게 변환한다 — 회전 두 번
- 3 시간 적분과 지구 곡률 — 60초 직진하면 11 m 떠오른다
- 4 한 스텝의 순서 — 다섯 단계

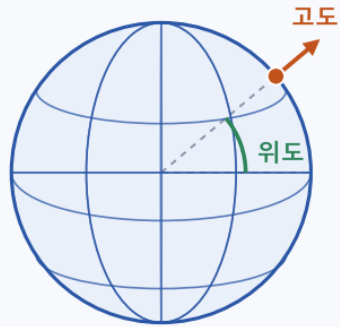
이 발표의 약속 : 각도는 도($^{\circ}$) 로 말하고 프로그램 안에서는 라디안. 거리는 m, 시간은 s.

01. 좌표계 넷, 그리고 왜 ECEF 인가

입력·출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체

LLA 위도 · 경도 · 고도

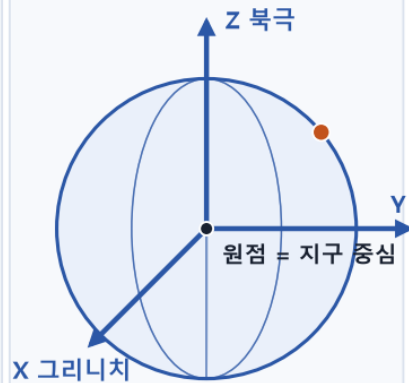
"지도 주소" — 사람이 읽는 자리



경도는 자오선에서 잰 각
고도는 타원체 면에서 잰 높이

ECEF 지구 중심 xyz

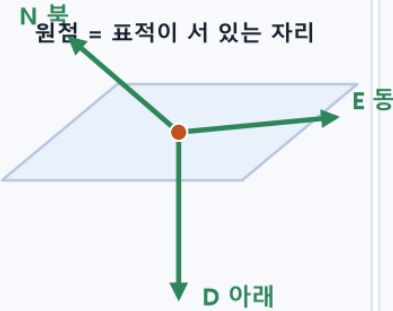
"지구에 박힌 자" — 계산하는 자리



지구와 함께 도는 직교좌표 (m)

NED 북 · 동 · 아래

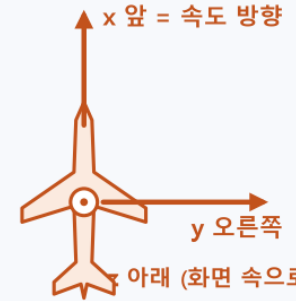
"내 발밑 나침반" — 수평면 기준



표적이 움직이면 같이 따라간다

동체 앞 · 오른쪽 · 아래

"표적의 몸" — 속도가 적힌 자리



속력 V 는 늘 x 축 위에만 있다

이번 과제에서의 역할

LLA

초기값을 받고, 매 스텝 결과를 적는 형식

ECEF

위치와 속도를 600번 갱신하는 자리

NED

표적 자리의 수평면. 자세각의 기준이자 속도 방향을 만드는 자리

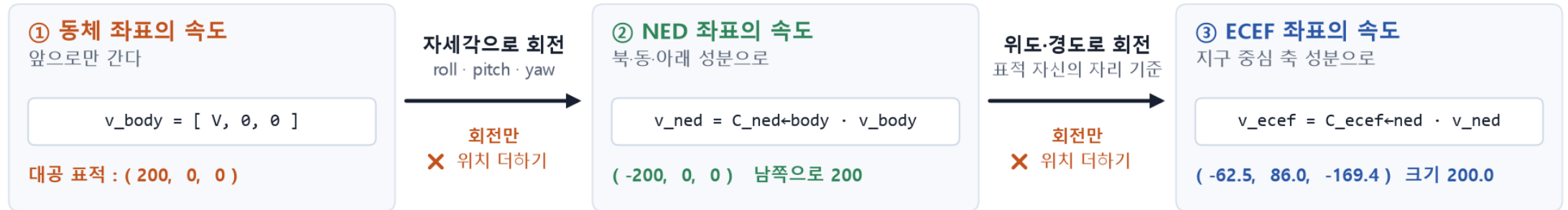
동체

속력 V 가 적힌 자리. 앞으로 V , 옆·아래는 0

왜 ECEF 인가 : 위도 $1^\circ = 111 \text{ km}$, 경도 $1^\circ = 94 \text{ km}$ (32°N) — 각도에 미터를 더할 수 없다. ECEF 는 세 축이 미터라 $p + v \cdot \Delta t$ 한 줄 플랫폼과 무관한 절대 좌표라, 나중에 배가 움직여도 표적 궤적이 흔들리지 않는다.

01. 위치와 속도는 다르게 변환한다 — 회전 두 번

위치는 원점이 있어야 말이 되고, 속도는 화살표 하나면 된다. 그래서 속도는 회전만



속도는 화살표라서 두 번 다 회전만 한다. ③ 에서 위도·경도는 플랫폼이 아니라 표적이 지금 있는 자리의 값이다.

위치 : $\mathbf{p}_{\text{new}} = \mathbf{R} \cdot \mathbf{p} + \mathbf{t}$

원점이 다르면 숫자가 다르다.
회전하고 원점 차이만큼 옮긴다 (과제 2).

속도 : $\mathbf{v}_{\text{new}} = \mathbf{R} \cdot \mathbf{v}$

어디에 그러도 같은 화살표.
축이 돌아간 만큼만 돌린다. $\mathbf{t}$ 를 더하면 속도가 아니게 된다.

검산

결과 벡터의 크기는 항상 V (200.000).
플랫폼 위치를 잘못 더하면 6,372 km/s.

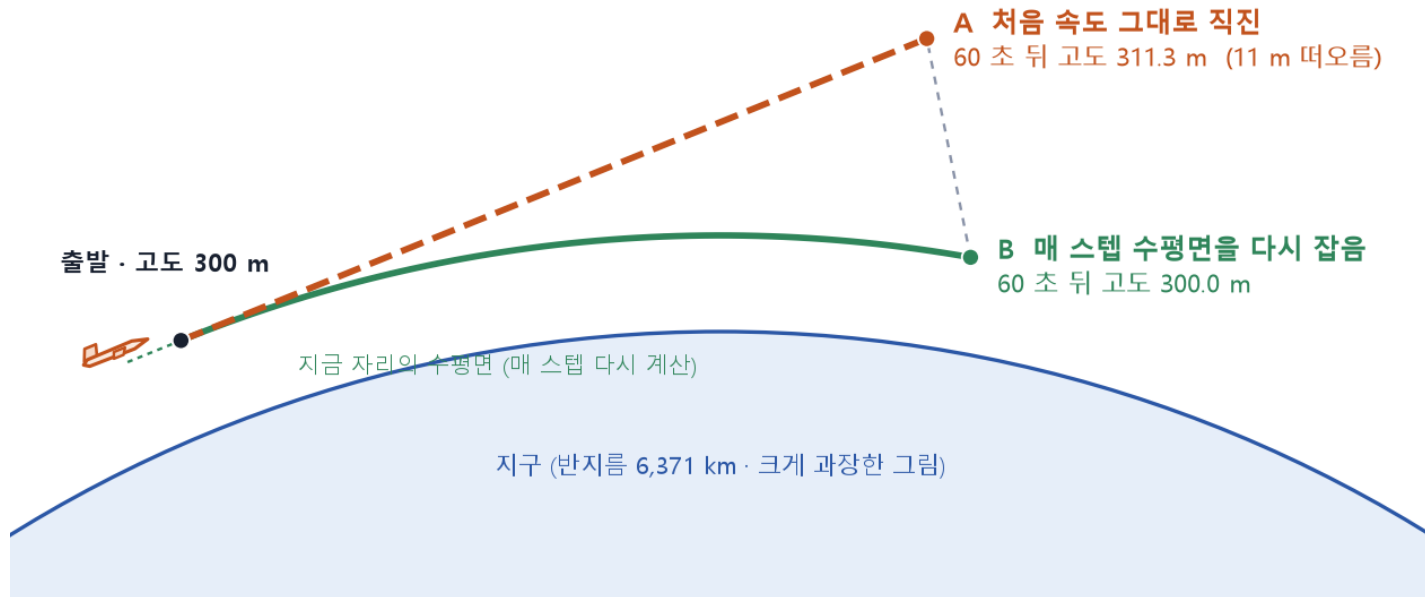
과제의 "추가 고민" (동체 속도 $\mathbf{V}_{\text{heading}} \rightarrow \text{ECEF}$) 의 답 : 자세각으로 한 번, 표적 자신의 위경도로 한 번, 평행이동은 없다

참고 함수 $\text{f_Trans_Ned_To_Ecef}$ 는 플랫폼 위치를 더하므로, 속도를 넣을 때는 그 인자에 0 을 준다.

01. 시간 적분과 지구 곡률

새 위치 = 지금 위치 + 속도 × 0.1 초. 단, 매 스텝 수평면을 다시 잡지 않으면 60초에 11 m 떠오른다

왜 떠오르나 : 지구가 둥글어서 접선 방향으로 12 km 가면 땅이 11 m 아래로 내려간다 ($d^2 / 2R$)
그래서 매 스텝 표적의 현재 위경도로 NED 수평면을 다시 잡고, 그 위에서 속도 방향을 다시 만든다



시간 적분 : $p(t+\Delta t) = p(t) + v(t) \cdot \Delta t$

ECEF x, y, z 에 각각. $\Delta t = 0.1$ s, 600 스텝.
한 스텝 안에서 속도가 일정하니 이 한 줄이면 정확.

실험 (대공 표적 60초 직진)	60초 뒤 고도
A 처음 ECEF 속도를 60초 내내 그대로	311.3 m
B 매 스텝 현재 위경도로 NED 를 다시 잡음	300.0 m

왜 11 m 인가

접선을 따라 d 만큼 가면 땅은 $d^2 / 2R$ 만큼 내려간다.
 $12,000^2 / (2 \times 6,371,000) = 11.3$ m. 실험값과 일치.

그래서 5단계 LLA 변환은 출력용만이 아니라 다음 스텝의 수평면(NED 축) 을 만드는 기준이다

위경도를 구해야 NED 축이 나오고, NED 축이 있어야 속도 방향이 나온다. 이 순서를 지키면 수평 비행이 수평으로 유지된다.

01. 한 스텝의 순서 — 다섯 단계

표적 하나가 0.1초 뒤로 가는 일. 이 순서를 600번, 표적 수만큼 반복한다

표적 하나의 한 스텝 (0.1 초)

왼쪽부터 순서대로. ① ② 는 기동이 없으면 건너뛰고, ③ ④ ⑤ 는 매 스텝 반드시 한다.



① 에 필요한 것

기동 목록, 지금 시각 t

② 에 필요한 것

각속도 $\omega = n \cdot g / V$, Δt

③ 에 필요한 것

자세각 3개, 표적 위경도

④ 에 필요한 것

ECEF 위치·속도, $\Delta t = 0.1\text{ s}$

⑤ 에 필요한 것

ECEF → LLA 변환 함수

표적 10개 × 600 스텝 = 6,000번. 순서가 중요한 건 ③ ④ ⑤ — 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면

기동 모델

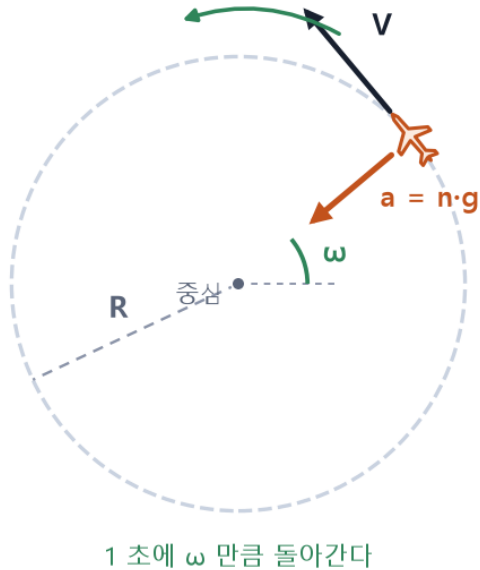
1 하중배수와 원운동 공식

2 세 축과 기동 시간표

약속 : 기동 중에도 속력은 바뀌지 않는다. 방향만 바뀐다.

02. 하중배수와 원운동 공식

과제의 Gravity Value 는 "몇 g 로 도는가". 속력은 그대로, 방향만 바뀐다



등속 원운동 공식

구심가속도
 $a = V^2 / R = n \cdot g$

선회 반경
 $R = V^2 / (n \cdot g)$

각속도
 $\omega = V / R = n \cdot g / V$

$g = 9.80665 \text{ m/s}^2$
 V 는 변하지 않는다

표적	하중배수	각속도	선회 반경	비고
대공 200 m/s	1 g	2.81 °/s	4,079 m	180° 에 64 s
대공 200 m/s	3 g	8.43 °/s	1,360 m	180° 에 21 s
대공 200 m/s	5 g	14.05 °/s	816 m	180° 에 13 s
대함 30 m/s	0.1 g	1.87 °/s	918 m	90° 에 48 s

감 잡기 : 1 g 서 있을 때, 0.3 g 승용차 커브, 3 g 롤러코스터 바닥·전투기 급선회, 9 g 전투기 한계

가속도가 속도에 수직이라 속력은 안 변하고 방향만 변한다.
 각속도가 속력에 반비례 — 배 기동에는 0.1 g 같은 작은 값을 넣어야 그럴듯하다.

프로그램에서는 한 줄 : $\omega = n \cdot g / V$ 를 구해 놓고, 매 스텝 자세각에 $\omega \times 0.1 \text{ s}$ 를 더한다 (3 g 면 한 스텝에 0.84°)

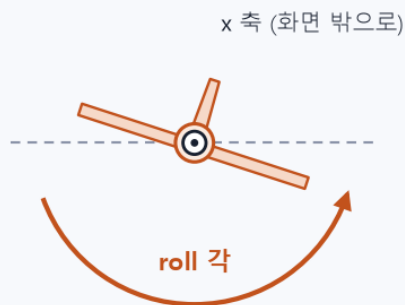
$$a = V^2/R = n \cdot g \rightarrow R = V^2/(n \cdot g), \omega = V/R = n \cdot g/V$$

02. 세 축과 기동 시간표

TurnType 은 동체의 어느 축을 돌리는가. 기동 하나 = { g, TurnType, StartTime, EndTime }, 표적마다 최대 30개

Roll (x 축 · 앞뒤 축)

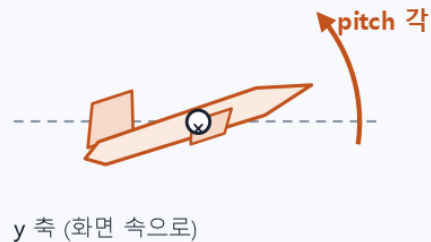
뒤에서 본 그림 · 날개가 기운다



속도 방향은 그대로

Pitch (y 축 · 날개 축)

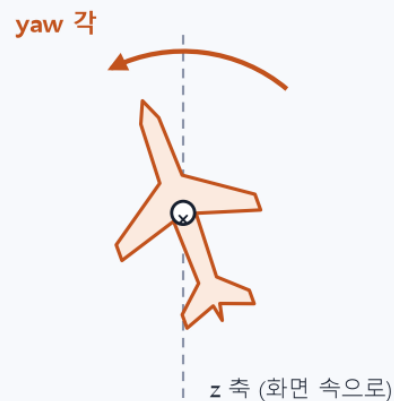
옆에서 본 그림 · 기수가 들린다



상승 · 강하

Yaw (z 축 · 위아래 축)

위에서 본 그림 · 기수가 돌아간다



좌우 선회

TurnType	축	효과
0	—	직진
1 Roll	x (앞뒤)	자세만 기움
2 Yaw	z (위아래)	좌우 선회
3 Pitch	y (날개)	상승 · 강하

시간표 규칙 셋

StartTime ≤ t < EndTime 이면 적용, 밖이면 직진.

부호 : n < 0 이면 반대 방향. 겹침 : 먼저 적힌 것. 끝난 뒤 : 자세 유지.

축을 돌린 뒤 속도가 어느 쪽인지는 따로 계산하지 않는다 — 자세각만 바뀌 두면 6장의 회전 두 번이 새 속도 방향을 만든다

과제 조건에는 기동 목록이 없다 → 발표용 예시 기동을 정해 넣었다 (대공 3 g 우선회 10~20 s, 2 g 기수 들기 35~40 s, 대함 -0.1 g 좌선회 20~50 s)

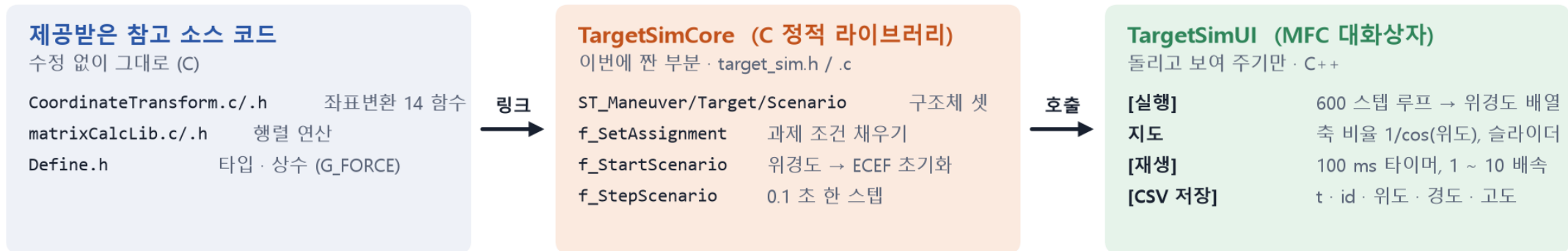
C 로 구현

- 1 프로그램 구성 — 제공 코드 · C 코어 · MFC 화면
- 2 표적 구조체 — target_sim.h
- 3 과제 조건 넣기 — f_SetAssignment
- 4 속도 변환 — f_BodyVelToEcef
- 5 한 스텝 — f_StepTarget
- 6 루프와 화면 — TargetSimUI

제공받은 좌표변환 코드는 한 글자도 고치지 않았다. MFC 는 계산을 하지 않는다.

03. 프로그램 구성

제공 코드는 무수정으로 링크, C 코어가 순서대로 부르고, MFC 는 돌려서 보여 주기만 한다



좌표변환은 전부 제공 코드가 하고, 코어는 그 함수를 순서대로 부르지만 한다. MFC 는 계산을 하지 않는다.

제공 코드 5 파일 = 원본 그대로

복사 후 바이트 단위 비교로 확인.
좌표변환은 이 코드만 부른다.

C 코어 250줄

구조체 셋 + 함수 다섯.
MSVC x64 · C17 · 경고 4단계 0건.

MFC 화면 (Part 2 CoordUI 와 같은 뼈대)

계산 없음. 코어를 600번 부르고 배열에 모아 그린다
· 지도 · 슬라이더 · 재생 · CSV 저장.

걸린 것 하나 : 제공 Define.h 의 #define BOOL bool 이 MFC 의 BOOL(int) 과 충돌 → 제공 코드는 두고, MFC 쪽에서 헤더를 읽은 직후 #undef BOOL

03. 표적 구조체 — target_sim.h

위 다섯 칸이 입력(사람이 적는 값), 아래 네 칸이 상태(프로그램이 갱신). 좌표 타입은 제공 코드의 구조체 그대로

표적 카드 #2

ST_Target

입력 (사람이 적는 칸)

초기 위치 위경도 · 고도로
32.12°N 126.0°E 300 m

동체 속도 앞 방향 하나로
200 m/s

초기 자세
roll 0° pitch 0° yaw 180°

기동 목록 (최대 30 개)

g	Type	Start	End
3	Yaw	10	20
2	Pitch	35	40
...			

초기화
→
한 번

상태 (프로그램이 갱신)

ECEF 위치
x, y, z [m]

ECEF 속도
vx, vy, vz [m/s]

현재 자세
roll, pitch, yaw

현재 위경도
출력용 · 매 스텝 변환



매 스텝 갱신 · 600 번
입력 칸은 건드리지 않는다

플랫폼도 속도 0 인 표적 카드

갱신 함수가 하나로 통일되고, 배가 움직이는 시나리오가 와도 카드 값만 바꾸면 된다. MAX_TARGET 10, MAX_MANEUVER 30.

```
typedef struct
{
    DOUBLE64    dGravity;        // 하중배수 [g]. 부호가 방향
    INT32        nTurnType;       // 0 없음 1 Roll 2 Yaw 3 Pitch
    DOUBLE64    dStartTime;      // [s]
    DOUBLE64    dEndTime;       // [s]
} ST_Maneuver;

typedef struct
{
    STRUCT_Coord_Lla    stInitLla;        /* 입력 */
    DOUBLE64            dVheading;        // 초기 위치 [rad, rad, m]
    STRUCT_Coord_Attitude stInitAtt;      // 동체 속도 [m/s]
    INT32               nManeuverCnt;     // 초기 자세 [rad]
    ST_Maneuver         astManeuver[MAX_MANEUVER]; // 기동 수
                                /* 상태 (ECEF 에서 갱신) */
    STRUCT_Coord_Rect    stPos;          // ECEF 위치 [m]
    STRUCT_Coord_Rect    stVel;          // ECEF 속도 [m/s]
    STRUCT_Coord_Attitude stAtt;         // 현재 자세 [rad]
    STRUCT_Coord_Lla     stLla;          // 현재 위경도 (출력용)
} ST_Target;

typedef struct
{
    ST_Target    stPlatform;        // 속도 0 인 표적 카드
    INT32        nTargetCnt;
    ST_Target    astTarget[MAX_TARGET]; // 10
    DOUBLE64    dSimTime;           // 60 s
    DOUBLE64    dDt;                // 0.1 s
} ST_Scenario;
```

03. 과제 조건 넣기 — f_SetAssignment

각도는 도로 받아 함수 안에서 라디안으로. 체크박스를 켜면 발표용 기동 예시가 붙는다

```
VOID f_SetAssignment(ST_Scenario *pstScn, INT32 bWithManeuver)
{
    memset(pstScn, 0, sizeof(*pstScn));
    pstScn->dSimTime = 60.0;
    pstScn->dDt = 0.1;
    pstScn->nTargetCnt = 2;

    /*      위도      경도      고도      속도      roll      yaw      pitch */
    f_SetTarget(&pstScn->stPlatform, 32.0, 126.0, 0.0, 0.0, 0.0, 0.0, 0.0);
    f_SetTarget(&pstScn->astTarget[0], 32.125, 126.03, 0.0, 30.0, 0.0, 270.0, 0.0); // 대함
    f_SetTarget(&pstScn->astTarget[1], 32.12, 126.0, 300.0, 200.0, 0.0, 180.0, 0.0); // 대공

    if (bWithManeuver) // 발표용 기동 예시
    {
        f_AddManeuver(&pstScn->astTarget[0], -0.1, TURN_YAW, 20.0, 50.0); // 대함 좌선회
        f_AddManeuver(&pstScn->astTarget[1], 3.0, TURN_YAW, 10.0, 20.0); // 대공 3 g 우선회
        f_AddManeuver(&pstScn->astTarget[1], 2.0, TURN_PITCH, 35.0, 40.0); // 대공 기수 들기
    }

    // f_SetTarget : 각도는 도로 받아 라디안으로 저장한다 (프로그램 안은 라디안)
    pstTgt->stInitLla.Lat = DEG2RAD(dLatDeg); ... pstTgt->stInitAtt.Yaw = DEG2RAD(dYawDeg);
}
```

	플랫폼	표적 1 대함	표적 2 대공
위도 · 경도	32.0 · 126.0	32.125 · 126.03	32.12 · 126.0
고도 m	0	0	300
속력 m/s	0	30	200
yaw	0	270 (서)	180 (남)

기동 예시 (화면의 체크박스)

대함 : -0.1 g yaw 20~50 s (음수 = 좌선회)

대공 : +3 g yaw 10~20 s, +2 g pitch 35~40 s

표적을 더 넣으려면 f_SetTarget 줄을 더 쓴다 (최대 10).

시간 60 s · 간격 0.1 s · 표적 2 개 — 과제 시뮬레이션 조건 그대로

03. 속도 변환 — f_BodyVelToEcef

6장의 회전 두 번이 열 줄. 참고 코드에서 조심할 점 두 가지가 이 화면에 다 있다

① 동체 → NED

f_Trans_Body_To_Ned 에 [V, 0, 0] 과 자세각.
인자 순서는 Roll, Yaw, Pitch (헤더에 적힌 대로).

② NED → ECEF

f_Trans_Ned_To_Ecef 의 가운데 셋이 플랫폼 위치.
속도는 벡터라 여기에 0. 위경도는 표적 자신의 것.

초기화의 함정

f_Trans_Lla_To_Ecef 는 (경도, 위도, 고도) 순서.
바꿔 놓으면 북위 54° 동경 212° 가 나온다.

검산 : $|v_{ecef}| = 200.000000 \text{ m/s}$

실수로 플랫폼 위치를 더하면 6,372 km/s

```
// 동체 속도 [V, 0, 0] 을 ECEF 속도로. 회전 두 번, 평행이동 없음
static STRUCT_Coord_Rect f_BodyVelToEcef(const ST_Target *pstTgt)
{
    STRUCT_Coord_Rect stNed;

    // 1) 동체 → NED : 자세각으로 회전 (인자 순서 Roll, Yaw, Pitch)
    stNed = f_Trans_Body_To_Ned(pstTgt->dVheading, 0.0, 0.0,
                                pstTgt->stAtt.Roll, pstTgt->stAtt.Yaw, pstTgt->stAtt.Pitch);

    // 2) NED → ECEF : 표적 자신의 위경도로 회전.
    // 속도는 벡터라 플랫폼 위치 인자(가운데 셋) 에 0
    return f_Trans_Ned_To_Ecef(stNed.x, stNed.y, stNed.z, 0.0, 0.0, 0.0,
                                pstTgt->stLla.Lat, pstTgt->stLla.Lon);
}

// 초기화 : 입력 칸 → 상태 칸. f_Trans_Lla_To_Ecef 는 인자 순서가 (경도, 위도, 고도)
static VOID f_InitTarget(ST_Target *pstTgt)
{
    pstTgt->stPos = f_Trans_Lla_To_Ecef(pstTgt->stInitLla.Lon,
                                        pstTgt->stInitLla.Lat, pstTgt->stInitLla.Alt);

    pstTgt->stAtt = pstTgt->stInitAtt;
    pstTgt->stLla = pstTgt->stInitLla;
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);
}
```

03. 한 스텝 — f_StepTarget

8장의 다섯 단계가 함수 하나. 주석 번호가 8장 번호와 같다

① ② 기동 판단 · 자세 갱신

for 문. 구간 안이면 $\omega \cdot dt$ 만큼 그 축을 돌리고 break (먼저 적힌 기동 우선). $+1e-6$ 은 0.1 을 600번 더한 부동소수점 찌꺼기 여유.

③ 속도 변환

매 스텝 다시 부른다. 자세가 안 바뀌어도 표적의 위경도가 바뀌니까.

④ 위치 적분

ECEF 세 성분에 $v \times dt$. 이 세 줄이 시간 적분.

⑤ LLA 변환

기록되고, 다음 스텝 ③ 의 수평면 기준이 된다 (지구 곡률 처리).

속력 0 인 플랫폼도 같은 함수를 지난다 — 기동 판단에 속력 0 이면 건너뛰는 조건이 있고, 속도가 0 이라 자리만 지킨다.

```
static VOID f_StepTarget(ST_Target *pstTgt, DOUBLE64 dTime, DOUBLE64 dDt)
{
    INT32 i;

    // 1) 기동 판단, 2) 자세 갱신 : 각속도  $\omega = n \cdot g / V$ 
    for (i = 0; i < pstTgt->nManeuverCnt; i++)
    {
        const ST_Maneuver *pstMan = &pstTgt->astManeuver[i];

        if ((dTime + 1e-6 >= pstMan->dStartTime) && (dTime + 1e-6 < pstMan->dEndTime)
            && (pstTgt->dVheading > 0.0))
        {
            DOUBLE64 dAng = pstMan->dGravity * G_FORCE / pstTgt->dVheading * dDt;

            if (pstMan->nTurnType == TURN_ROLL)    pstTgt->stAtt.Roll += dAng;
            else if (pstMan->nTurnType == TURN_YAW) pstTgt->stAtt.Yaw += dAng;
            else if (pstMan->nTurnType == TURN_PITCH) pstTgt->stAtt.Pitch += dAng;
            break;
            // 먼저 적힌 기동 하나만
        }
    }

    // 3) 속도 변환 (표적 자신의 현재 위경도 기준 NED)
    pstTgt->stVel = f_BodyVelToEcef(pstTgt);

    // 4) 위치 적분 (ECEF)
    pstTgt->stPos.x += pstTgt->stVel.x * dDt;
    pstTgt->stPos.y += pstTgt->stVel.y * dDt;
    pstTgt->stPos.z += pstTgt->stVel.z * dDt;

    // 5) LLA 변환. 출력용이자 다음 스텝의 NED 기준
    pstTgt->stLla = f_Trans_Ecef_To_Lla(pstTgt->stPos.x, pstTgt->stPos.y, pstTgt->stPos.z);
}
```

03. 루프와 화면 — TargetSimUI (MFC)

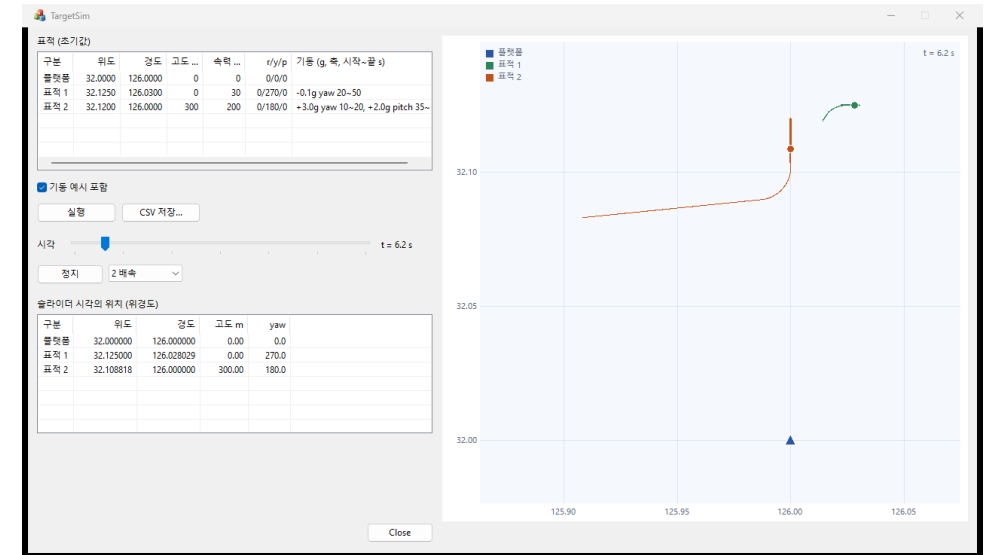
MFC 는 계산이 없다. 코어를 600번 부르고, 배열에 모으고, 지도에 그리고, 시각을 움직인다

```
// TargetSimUIDlg.cpp [실행] : 60 초를 0.1 초 간격으로 돌리며 매 스텝 위경도를 모은다
const int nSteps = (int)floor(m_stScn.dSimTime / m_stScn.dDt + 0.5); // 600

f_StartScenario(&m_stScn); // 위경도 → ECEF, 속도 초기화
record(); // t = 0
for (int k = 0; k < nSteps; k++)
{
    f_StepScenario(&m_stScn, k * m_stScn.dDt); // 플랫폼 + 표적 전부 한 스텝
    record(); // 위도 · 경도 · 고도 · yaw 를 배열에
}
m_map.SetPaths(&m_paths); // 지도에 궤적, 슬라이더 0 ~ 600
```

```
// MapView.cpp 위도 1 도가 경도 1 도보다 1/cos(위도) 배 길다 → 축 비율
const double dAsp = 1.0 / cos(dLatC * PI / 180.0);
if (dLatSpan * dAsp * dW / dH > dLonSpan) dLonSpan = dLatSpan * dAsp * dW / dH;
else dLatSpan = dLonSpan / dAsp * dH / dW;
```

```
// 재생 : 100 ms 타이머 한 틱에 배속(1.2·5·10) 만큼 스텝 → 1 배속 = 실시간
SetTimer(TIMER_PLAY, 100, nullptr);
```



2 배속 재생 중 (t = 6.2 s). 점의 짧은 선이 진행 방향.

CSV 저장

t, id, 위도, 경도, 고도, yaw 를 한 줄씩 — 과제가 요구한 위경도 출력. 뒤의 파이썬 그림은 이 파일로 그렸다.

지도의 축 비율 $1/\cos(\text{위도})$ 를 안 맞추면 선의 원이 타원으로 보인다. 재생은 1 배속이 실시간, 10 배속까지.

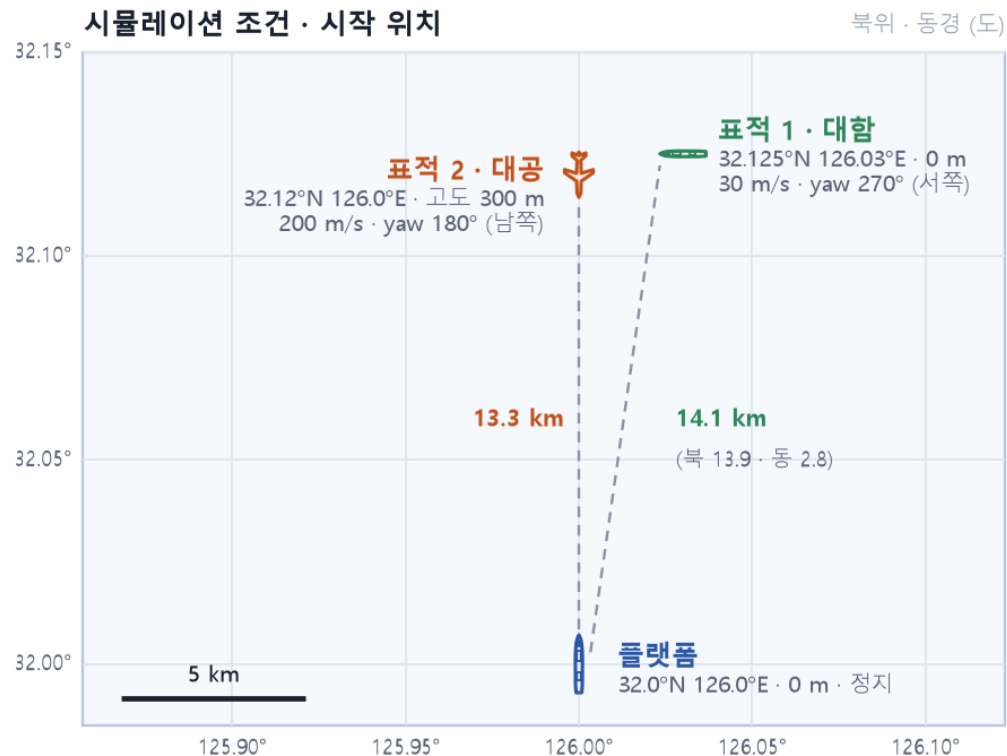
결과

- 1 시뮬레이션 조건
- 2 실행 화면
- 3 궤적과 고도
- 4 기동 시나리오
- 5 검증

화면의 값 = CSV 의 값 = 파이썬 그림의 값. 같은 코드가 만든 같은 숫자다.

04. 시뮬레이션 조건 — 플랫폼 하나, 표적 둘

대공 표적은 정면으로 접근하고, 대함 표적은 정북 방향을 가로지른다



항목	플랫폼	대함 표적 (1)	대공 표적 (2)
위도 · 경도	32.0°N 126.0°E	32.125°N 126.03°E	32.12°N 126.0°E
고도	0 m	0 m	300 m
속력 (동체 앞)	0 m/s	30 m/s	200 m/s
자세각 (r · y · p)	—	0° · 270° · 0° (서쪽)	0° · 180° · 0° (남쪽)
시간	60 s, 간격 0.1 s (600 스텝)		

플랫폼 기준으로 바꿔 보면

대공 : 정북 13.3 km, 고각 1.2° → 정확히 우리 쪽으로 온다 (정면 접근)

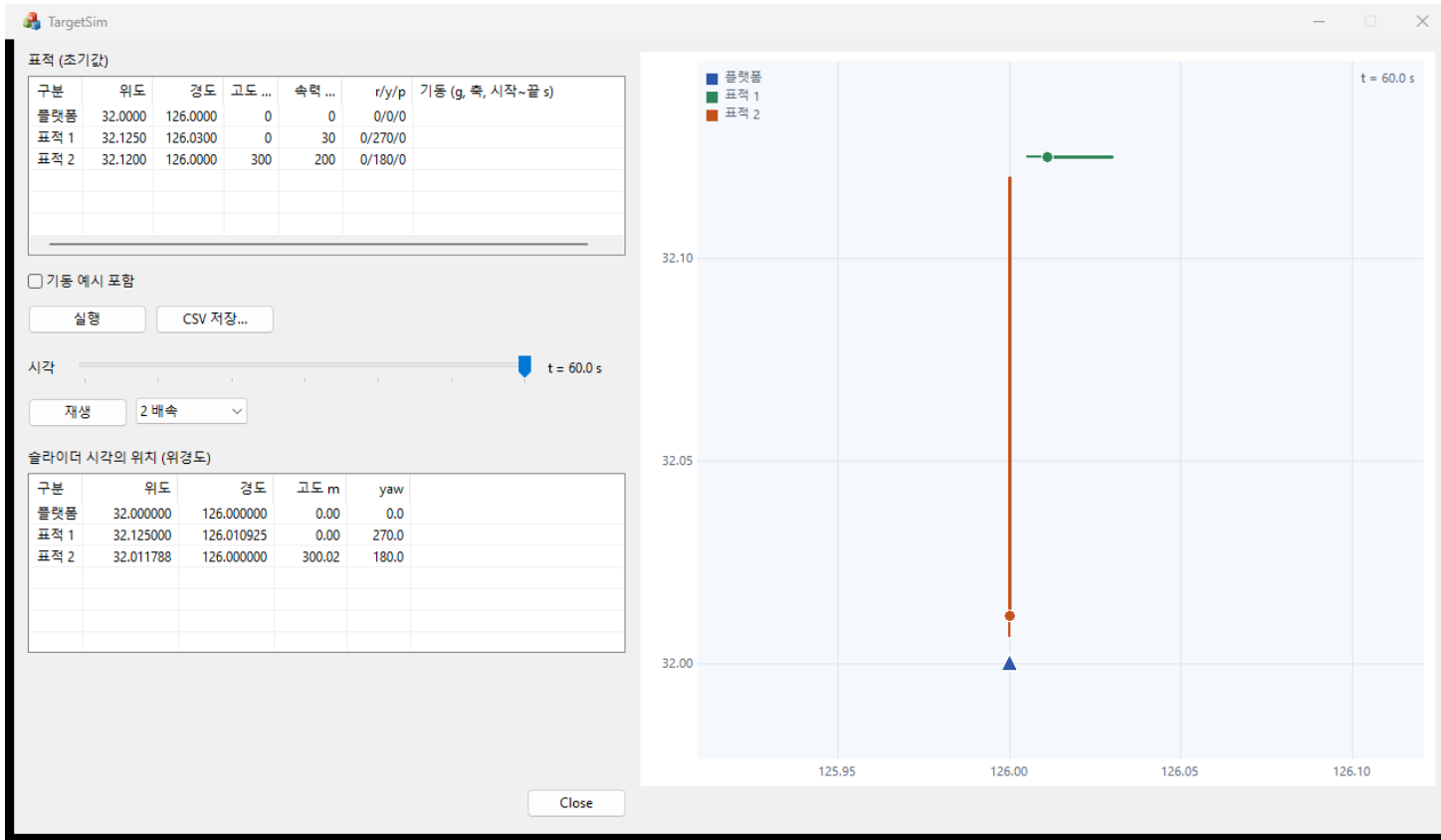
대함 : 북 13.9 km · 동 2.8 km, 거리 14.1 km → 서쪽으로 가며 정북을 가로지른다

두 표적의 출발점은 550 m · 2.8 km 차이로 아주 가깝다

같은 자리에서 출발해 하나는 남쪽으로 빠르게, 하나는 서쪽으로 천천히 — 그림에서 대비가 명확하도록 짜인 조건

04. 실행 화면 — 과제 조건, $t = 60\text{ s}$

왼쪽 위 표적 카드의 입력 칸, 오른쪽 궤적 지도, 왼쪽 아래 슬라이더 시각의 상태 칸



대공 표적 (주황)

60 s 뒤 32.011788°N, 126.0°E, 고도 300.02 m
 플랫폼에서 1,341 m 앞, 고각 1.2° → 12.9°
 정남으로 내려와 삼각형 바로 위에서 끝난다

대함 표적 (초록)

60 s 뒤 32.125°N, 126.010925°E
 서쪽으로 1.8 km. 선 길이 차이 = 속력 차이

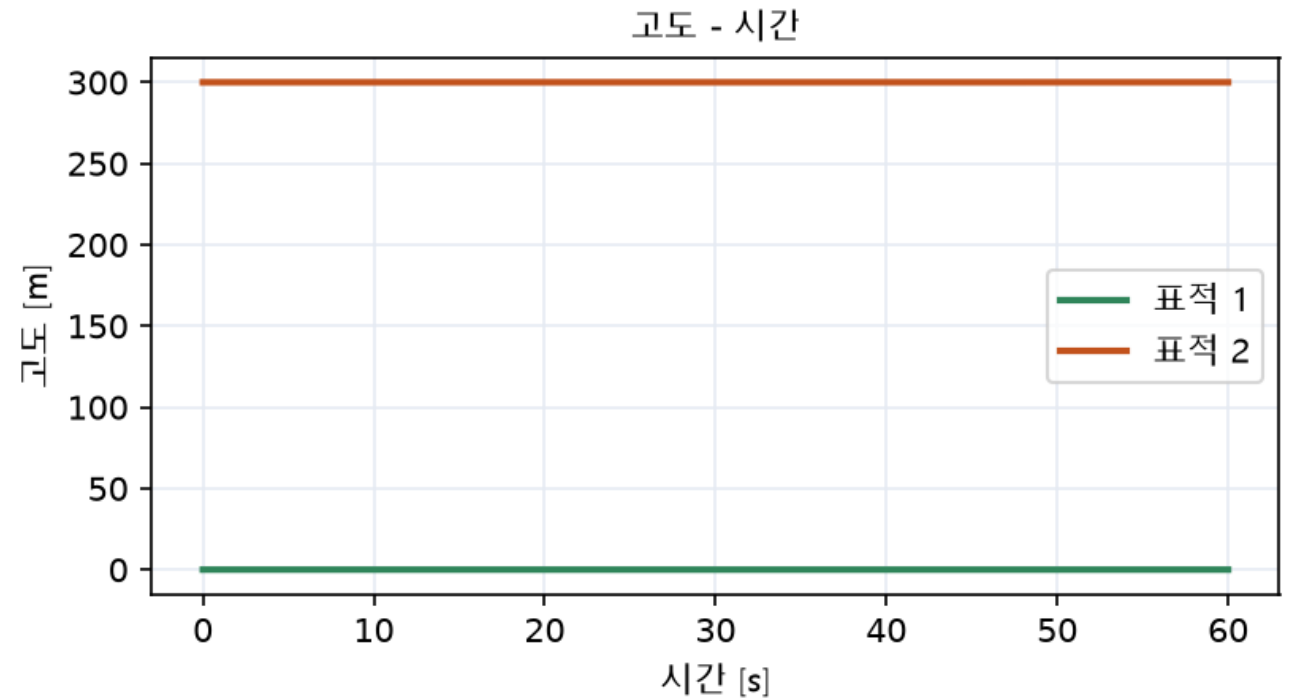
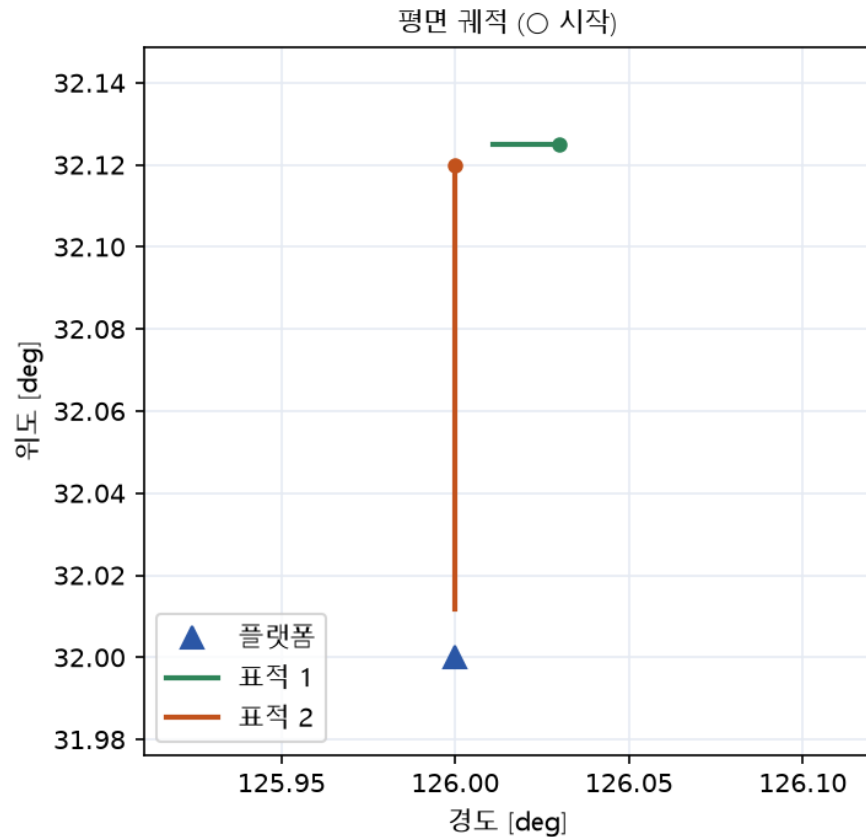
플랫폼 (파랑 삼각형)

32.0°N 126.0°E 에서 자리를 지킨다.
 같은 스텝 함수를 지나지만 속력이 0.

상태 칸의 값이 그대로 [CSV 저장] 으로 나간다 — 과제 요구 "시뮬레이션 결과를 LLA 로 출력"

04. 궤적과 고도 — CSV 를 파이썬으로

평면 궤적은 축 비율을 맞춰서, 고도는 평평해야 한다 (지구 곡률 처리의 증거)



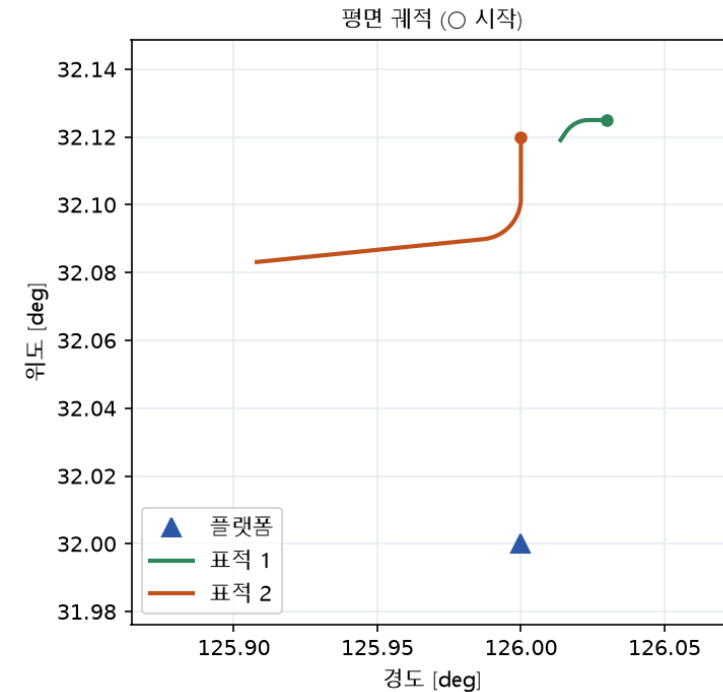
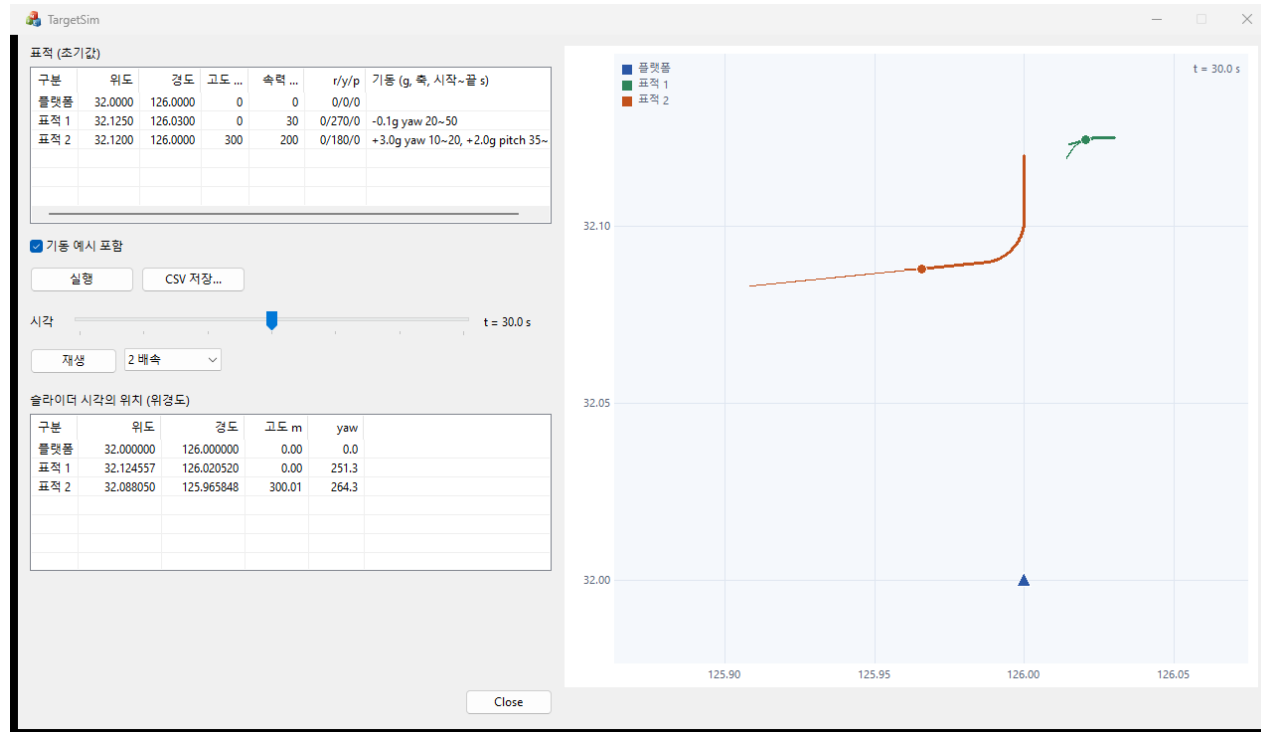
60 s 뒤 고도 300.019 m — 2 cm 의 정체

한 스텝에 20 m 접선으로 가서 $20^2 / 2R$ 씩 뜨고, 600번이면 1.9 cm. 이론값과 같다.

매 스텝 수평면을 다시 잡지 않았다면 이 선이 300 → 311 m 로 기울었을 것

04. 기동 시나리오 — 3 g 우선회, 기수 들기, 좌선회

체크박스를 켜고 실행. 왼쪽은 $t = 30$ s 화면, 오른쪽은 CSV 로 그린 궤적



대공 3 g 우선회 (10~20 s)

10초에 84.3° 돌아 서쪽으로.

궤적 세 점 외접원 1,359.5 m vs $V^2/(n \cdot g)$ 1,359.6 m

대공 2 g 기수 들기 (35~40 s)

5초에 28.1° 올라간 뒤 그 각으로 20초.

60 s 에 고도 2,428.7 m

대함 -0.1 g 좌선회 (20~50 s)

30초에 -56.2° , 남서쪽으로.

반경 917.8 m vs 이론 917.7 m

04. 검증 — 다섯 가지로 확인

좌표변환은 틀려도 그럴듯한 값이 나온다 (과제 2의 교훈). 저장한 CSV 로 계산했다

항목	어떻게 확인하나	기대값	결과
① 이동 거리	ECEF 위치 차이의 합 vs 속도 × 시간	$200 \times 60 = 12,000 \text{ m}$	12,000.000 m (차 1e-9 m)
② 속도 보존	매 스텝 $ v_{\text{ecef}} $ vs 입력 속도	200 m/s	최소·최대 200.000000 (편차 1e-14)
③ 고도 유지	수평 비행 60 s 뒤 고도	300 m	300.019 m (이론 +1.9 cm)
④ 선회 반경	기동 시작·중간·끝 세 점의 외접원 vs $V^2/(n \cdot g)$	3 g : 1,359.6 m	1,359.5 m (−0.01 %)
⑤ 위경도 왕복	LLA → ECEF → LLA → ECEF 거리	1 mm 미만	3e-5 m

왜 미리 정하나

코드가 내놓을 값을 먼저 알고 있어야 코드가 맞는지 알 수 있다.

①②③⑤ 는 개념편에서 손계산한 값과 같고, ④ 는 기동 코드가 붙은 뒤 잦다.

한 번 틀리면 바로 드러나는 실수들

경도·위도 인자 순서 바꿈 → 북위 54° 동경 212°. 속도에 플랫폼 위치 더함 → 6,372 km/s.

수평면을 한 번만 잡음 → 고도 311 m.

다섯 칸이 채워졌으니 궤적이 "그림" 이 아니라 "결과" 다

정리

- 1 과제 3 = 과제 2의 변환 함수 + 시간 루프 + 기동 모델. 제공 코드는 한 줄도 안 고쳤다
- 2 위치는 회전 + 평행이동, 속도는 회전만. 속도에 위치를 더하면 안 된다
- 3 새 위치 = 지금 위치 + 속도 $\times$ 0.1 초. ECEF 에서 더한다
- 4 지구는 둥글다. 매 스텝 표적 자리의 수평면을 다시 잡아야 고도가 유지된다
- 5 기동은 원운동. $\omega = n \cdot g / V$, $R = V^2 / (n \cdot g)$. 코드가 0.01 % 로 재현했다
- 6 표적 카드 = 입력 칸 + 상태 칸. 플랫폼도 카드 한 장
- 7 검증 없는 궤적은 그림일 뿐. 거리 · 속력 · 고도 · 반경 · 왕복 다섯 가지로 확인했다

질문 환영합니다 코드 · 문서 : [radar-sw-study / Part3_TargetSim](#)

부록 A. 참고 코드의 함수와 함정

제공된 CoordinateTransform.c 에서 이번에 쓴 것과, 컴파일해서 확인한 주의점

함수	이번 과제의 용도	주의
f_Trans_Lla_To_Ecef (Lon, Lat, Alt)	초기 위치 입력	경도가 첫째 인자. 바꾸면 북위 54° 동경 212°
f_Trans_Ecef_To_Lla (X, Y, Z)	매 스텝 출력 + 다음 수평면 기준	5회 고정 반복. 우리 위치에서 왕복 오차 3e-5 m
f_Trans_Body_To_Ned (... , Roll, Yaw, Pitch)	동체 속력 → NED 속도	인자 순서가 Roll, Yaw, Pitch
f_Trans_Ned_To_Ecef (... , Pf_X, Pf_Y, Pf_Z, Lat, Lon)	NED 속도 → ECEF 속도	속도에는 Pf_X·Y·Z = 0. 아니면 6,372 km/s
Ant 계열 4개 · Enu 계열 2개 · PfCompensation · Ecef_To_Ned	사용 안 함	표적을 만드는 쪽이라 안테나 좌표계가 없다

각도는 전부 라디안 (DEG2RAD 매크로는 target_sim.h 에 정의) · Body → NED 행렬은 Rz(yaw)·Ry(pitch)·Rx(roll) 과 소수점 6자리까지 일치 (과제 2 와 같은 규약)

그 밖에 : 한글 주석이 UTF-8 이라 MSVC /utf-8 옵션 (vcxproj 에 포함). Define.h 의 #define BOOL bool 은 MFC 와 충돌 → MFC 쪽에서 #undef. G_FORCE = 9.80665 는 Define.h 에 있다.

부록 B. 용어 정리

발표에 나온 말들. 질문이 들어오면 이 표를 보면서 답한다

용어	뜻	이번 과제에서
하중배수 n (g)	안쪽으로 당기는 가속도가 중력가속도의 몇 배인가	기동의 Gravity Value. $a = n \cdot g$
각속도 ω	1초에 몇 도(라디안) 도는가	$\omega = n \cdot g / V$. 매 스텝 자세각 += $\omega \times 0.1 \text{ s}$
오일러 적분	새 값 = 지금 값 + 변화율 $\times$ 시간 간격	$p \leftarrow p + v \times \Delta t$. 스텝 안 속도 일정이라 정확
자세각 roll · pitch · yaw	동체 축이 수평면(NED)에 대해 돌아간 세 각	표적의 진행 방향. yaw 180° = 남쪽
ECEF	지구 중심 원점, 지구와 함께 도는 직교좌표 [m]	위치·속도를 갱신하는 자리
NED	한 지점의 북 · 동 · 아래 수평 좌표	표적 자신의 자리 기준. 매 스텝 다시 잡는다
타원체고	WGS-84 타원체 면에서 수직으로 잰 높이	과제의 "고도". 해발고도와 약 20 m 차이
회전행렬 R	한 좌표계의 축을 다른 좌표계 축으로 돌려 읽는 3×3	위치엔 $R \cdot p + t$, 속도엔 $R \cdot v$

이상입니다. 질문 환영합니다. 코드 · 문서 : [radar-sw-study / Part3_TargetSim](#)